

This is a copy of a shared ChatGPT chat. It won't be added to memory.

Report conversation

Uploaded a file

Uploaded a file

Uploaded a file

Uploaded a file

Uploaded a file

Uploaded a file

asks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Fix puck–paddle collision

The puck does not consistently respond correctly when it collides with a paddle, particularly at higher speeds or when it approaches at an angle. Fix the collision logic so the puck reliably rebounds from the paddles at different speeds and angles without passing through or getting stuck inside them.

Task 2: Implement match scoring

Add a scoring system where a player earns a point when the puck enters the opponent's goal. Display both players' scores during play, and determine the winner based on the final score.

Task 3: Implement a 30-second match timer

Add a 30-second countdown timer to the match. Display the remaining time during gameplay. When the timer reaches zero, stop the match and determine the result based on the scores. If both scores are equal, display a draw.

Task 4: Implement puck reset after scoring

After a goal is scored, reset the puck to the center of the table and prepare it for the next point. Make sure the puck's velocity, direction, and any other relevant state are correctly reset so each new point starts cleanly.

Expected Behavior

The puck bounces realistically off walls and paddles at any angle, and never gets stuck vibrating inside a paddle.

A point is scored only when the puck fully passes through the goal gap - hitting the wall elsewhere bounces the puck back normally.

After every goal, the puck immediately continues play from the center - it should never sit motionless.

The match runs for exactly 30 seconds. When time runs out, the match ends and shows the correct winner - or "Draw" if the score is tied.

i will be providing 4 tasks in separate prompts with fixes to be made in each task
provide me with the complete updated file(s) and the exact changes each time.

Task 1 complete — puck–paddle collision

I changed 2 files:

puck.py
collisions.py

game_engine.py, paddle.py, ai.py, and renderer.py do not need changes for Task 1. The existing update order already moves the puck and then invokes paddle collision handling.
game_engine

The original collision only checked the puck's final position and inverted vx, which is why angled impacts and high-speed movement were unreliable.
collisions

Exact changes

Added previous-frame puck position

Puck now stores previous_x and previous_y.

move() records the old position before moving.

Added swept collision detection

The entire line segment travelled by the puck during a frame is tested against the paddle.
This prevents a fast puck from moving completely through a paddle between two collision checks.

Changed collision response from $vx = -vx$ to vector reflection

The collision normal is calculated from the paddle to the puck.

Velocity is reflected using the normal, so angled impacts behave correctly.

Speed magnitude is preserved.

Added positional correction

After collision, the puck is placed just outside the paddle.

This prevents it from remaining embedded in the paddle and repeatedly vibrating.

Only reflect when moving toward the paddle

Prevents a puck that has already been pushed out from being reflected again on the next collision check.

Handles existing overlap

If the puck is already inside/intersecting a paddle, it is separated before continuing.

No scoring, timer, or reset behavior was added yet

Those remain for Tasks 2–4 as requested.

Validation

I ran syntax compilation across all six uploaded Python files and targeted collision tests covering:

high-speed tunneling through a paddle

angled collision

preservation of puck speed after reflection

existing puck/paddle overlap

missed paddle collision

ensuring the puck does not immediately collide again after being separated

All Task 1 collision tests passed.

Complete updated puck.py

```
"""
```

Puck: the disc players are trying to hit into the opponent's goal.

```
"""
```

```
class Puck:
```

```
    def __init__(self, x, y, radius=12):
```

```
        self.x = x
```

```
        self.y = y
```

```
        self.radius = radius
```

```
        self.vx = 0.0
```

```
        self.vy = 0.0
```

```
        # Keep the position from the start of the current frame so collision
```

```
        # handling can test the puck's entire path instead of only its final
```

```
        # position. This prevents high-speed tunneling through paddles.
```

```
        self.previous_x = x
```

```
        self.previous_y = y
```

```
    def move(self):
```

```
        self.previous_x = self.x
```

```
        self.previous_y = self.y
```

```
        self.x += self.vx
```

```
        self.y += self.vy
```

```
    def bounce_off_walls(self, height, margin):
```

```
        """Bounce off the top and bottom walls only - left/right are handled
        separately by the game engine, since they contain the goals."""
```

```

        if self.y - self.radius < margin:
            self.y = margin + self.radius
            self.vy = -self.vy
        elif self.y + self.radius > height - margin:
            self.y = height - margin - self.radius
            self.vy = -self.vy
Complete updated collisions.py
"""
collisions: puck-vs-paddle collision handling.
"""

```

```

import math

```

```

COLLISION_EPSILON = 1e-6

```

```

def _reflect_velocity(puck, normal_x, normal_y):
    """Reflect the puck's velocity across the supplied collision normal."""
    velocity_dot_normal = puck.vx * normal_x + puck.vy * normal_y

    # Only reflect while moving into the paddle. This prevents the positional
    # correction from causing repeated bounces while the puck is leaving it.
    if velocity_dot_normal < 0:
        puck.vx -= 2 * velocity_dot_normal * normal_x
        puck.vy -= 2 * velocity_dot_normal * normal_y
        return True

    return False

```

```

def handle_paddle_collision(puck, paddle):
    """
    Detect and resolve a puck/paddle collision.

```

Both objects are treated as circles. The puck's swept path from its previous position to its current position is checked, so a fast puck cannot pass through a paddle between frames.

When a collision is found, the puck is placed just outside the paddle and its velocity is reflected across the collision normal. This handles angled impacts correctly and prevents the puck from becoming embedded in the paddle.

Returns True if a collision was handled this frame.

"""

```
combined_radius = puck.radius + paddle.radius
combined_radius_sq = combined_radius * combined_radius
```

```
start_x = getattr(puck, "previous_x", puck.x)
start_y = getattr(puck, "previous_y", puck.y)
end_x = puck.x
end_y = puck.y
```

```
# Handle an overlap at the current position first. This also covers a
# puck that was already embedded in the paddle before this function ran.
dx = end_x - paddle.x
dy = end_y - paddle.y
distance_sq = dx * dx + dy * dy
```

```
if distance_sq <= combined_radius_sq:
    distance = math.sqrt(distance_sq)
```

```
if distance > COLLISION_EPSILON:
```

```
    normal_x = dx / distance
    normal_y = dy / distance
```

```
else:
```

```
    # If the centers coincide, use the opposite of the puck's motion
    # as the normal so the puck is pushed back along its incoming path.
```

```
    speed = math.hypot(puck.vx, puck.vy)
```

```
    if speed > COLLISION_EPSILON:
```

```
        normal_x = -puck.vx / speed
        normal_y = -puck.vy / speed
```

```
    else:
```

```
        normal_x, normal_y = 1.0, 0.0
```

```
puck.x = paddle.x + normal_x * (combined_radius + COLLISION_EPSILON)
```

```
puck.y = paddle.y + normal_y * (combined_radius + COLLISION_EPSILON)
```

```
_reflect_velocity(puck, normal_x, normal_y)
```

```
return True
```

```
# Swept circle-vs-circle test. Solve for the first point where the puck's
# center reaches the sum of the two radii from the paddle center.
```

```
segment_x = end_x - start_x
```

```
segment_y = end_y - start_y
```

```
relative_start_x = start_x - paddle.x
```

```
relative_start_y = start_y - paddle.y
```

```
segment_length_sq = segment_x * segment_x + segment_y * segment_y
if segment_length_sq <= COLLISION_EPSILON:
    return False
```

```
a = segment_length_sq
b = 2.0 * (relative_start_x * segment_x + relative_start_y * segment_y)
c = (
    relative_start_x * relative_start_x
    + relative_start_y * relative_start_y
    - combined_radius_sq
)
```

```
discriminant = b * b - 4.0 * a * c
if discriminant < 0:
    return False
```

```
sqrt_discriminant = math.sqrt(max(0.0, discriminant))
first_t = (-b - sqrt_discriminant) / (2.0 * a)
second_t = (-b + sqrt_discriminant) / (2.0 * a)
```

```
collision_t = None
for t in (first_t, second_t):
    if 0.0 <= t <= 1.0:
        collision_t = t
        break
```

```
if collision_t is None:
    return False
```

```
contact_x = start_x + segment_x * collision_t
contact_y = start_y + segment_y * collision_t
```

```
normal_x = contact_x - paddle.x
normal_y = contact_y - paddle.y
normal_length = math.hypot(normal_x, normal_y)
```

```
if normal_length <= COLLISION_EPSILON:
    speed = math.hypot(puck.vx, puck.vy)
    if speed <= COLLISION_EPSILON:
        return False
    normal_x = -puck.vx / speed
    normal_y = -puck.vy / speed
else:
    normal_x /= normal_length
```

```

    normal_y /= normal_length

    # Remove all overlap at the collision point before reflecting. The small
    # epsilon keeps floating-point error from leaving the puck intersecting
    # the paddle on the next frame.
    puck.x = paddle.x + normal_x * (combined_radius + COLLISION_EPSILON)
    puck.y = paddle.y + normal_y * (combined_radius + COLLISION_EPSILON)

    _reflect_velocity(puck, normal_x, normal_y)
    return True

```

The resulting collision behavior now uses the puck's actual travel path and the paddle's collision normal rather than simply reversing horizontal velocity. The existing paddle itself remains a clamped circular object as before.

paddle

Task 1 is isolated and complete. For your next prompt, I'll treat it as Task 2 and build on this version rather than redoing Task 1.

Sources

Task 3 should build directly on your successful Task 2 version. The changes are limited to game_engine.py and renderer.py; keep your Task 1 puck.py and collisions.py unchanged.

The existing engine currently has no match-state or elapsed-time tracking, and update() always advances the puck/AI.

game_engine The renderer already has the drawing separation we need for adding the timer and final-result banner.

renderer

Exact Task 3 changes
game_engine.py

Added:

```

MATCH_DURATION = 30.0
self.match_start_time
self.match_time_remaining
self.match_over
self.result
Timer calculation using pygame.time.get_ticks()
is_match_over()
get_winner() now returns the final result once the match ends
update() stops all gameplay once the timer reaches zero

```

Final result is:
"Player Wins!"
"Computer Wins!"
"Draw"

The timer is based on actual elapsed time rather than counting frames, so changing the game's FPS does not make the match last longer or shorter.

renderer.py

Added:

draw_timer()
draw_result()
Timer display during gameplay
Final result display after 30 seconds

The score remains visible after the match ends.

Complete updated game_engine.py
"""

GameEngine: owns the puck, both paddles, and the computer AI, and runs one frame's worth of game logic.

Task 3 adds a 30-second match timer and match-ending result handling.
"""

import random
import pygame

from game.puck import Puck
from game.paddle import Paddle
from game.ai import ComputerAI
from game.collisions import handle_paddle_collision
from game.renderer import WIDTH, HEIGHT, MARGIN, GOAL_TOP, GOAL_BOTTOM

PLAYER_SPEED = 6
PUCK_RADIUS = 12
PADDLE_RADIUS = 28
INITIAL_PUCK_SPEED = 4.5

MATCH_DURATION = 30.0

```

class GameEngine:
    def __init__(self):
        self.puck = Puck(WIDTH / 2, HEIGHT / 2, PUCK_RADIUS)
        self._launch_puck()

        self.player = Paddle(
            x=WIDTH * 0.15,
            y=HEIGHT / 2,
            radius=PADDLE_RADIUS,
            min_x=MARGIN + PADDLE_RADIUS,
            max_x=WIDTH / 2 - PADDLE_RADIUS,
            min_y=MARGIN + PADDLE_RADIUS,
            max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
        )

        self.computer = Paddle(
            x=WIDTH * 0.85,
            y=HEIGHT / 2,
            radius=PADDLE_RADIUS,
            min_x=WIDTH / 2 + PADDLE_RADIUS,
            max_x=WIDTH - MARGIN - PADDLE_RADIUS,
            min_y=MARGIN + PADDLE_RADIUS,
            max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
        )

        self.ai = ComputerAI()

        # Task 2: scores.
        self.player_score = 0
        self.computer_score = 0

        # Task 3: match timer/state.
        self.match_start_time = pygame.time.get_ticks()
        self.match_time_remaining = MATCH_DURATION
        self.match_over = False
        self.result = None

    def _launch_puck(self):
        angle_choices = [0.3, 0.6, -0.3, -0.6]
        direction = random.choice([-1, 1])
        vy_factor = random.choice(angle_choices)

        self.puck.vx = INITIAL_PUCK_SPEED * direction
        self.puck.vy = INITIAL_PUCK_SPEED * vy_factor

```

```

def handle_input(self, keys_pressed):
    # Do not allow player movement after the match has ended.
    if self.match_over:
        return

    dx = dy = 0

    if keys_pressed[pygame.K_UP]:
        dy -= PLAYER_SPEED

    if keys_pressed[pygame.K_DOWN]:
        dy += PLAYER_SPEED

    if keys_pressed[pygame.K_LEFT]:
        dx -= PLAYER_SPEED

    if keys_pressed[pygame.K_RIGHT]:
        dx += PLAYER_SPEED

    self.player.move_by(dx, dy)

def update(self):
    # Once the match has ended, the entire game state is frozen.
    if self.match_over:
        return

    self._update_timer()

    # The timer may have expired during _update_timer().
    if self.match_over:
        return

    self.ai.update(self.computer, self.puck)

    self.puck.move()
    self.puck.bounce_off_walls(HEIGHT, MARGIN)

    # Task 1 collision handling remains unchanged.
    handle_paddle_collision(self.puck, self.player)
    handle_paddle_collision(self.puck, self.computer)

    self._handle_goals()

```

```

def _update_timer(self):
    """
    Update the remaining match time using real elapsed time.

    pygame.time.get_ticks() is used instead of frame counting so the
    match lasts 30 real seconds regardless of the frame rate.
    """
    current_time = pygame.time.get_ticks()

    elapsed_seconds = (
        current_time - self.match_start_time
    ) / 1000.0

    self.match_time_remaining = max(
        0.0,
        MATCH_DURATION - elapsed_seconds,
    )

    if self.match_time_remaining <= 0.0:
        self.match_time_remaining = 0.0
        self._end_match()

def _end_match(self):
    """Stop the match and determine its final result."""
    if self.match_over:
        return

    self.match_over = True

    # Stop the puck so nothing continues moving visually after time expires.
    self.puck.vx = 0.0
    self.puck.vy = 0.0

    if self.player_score > self.computer_score:
        self.result = "Player Wins!"
    elif self.computer_score > self.player_score:
        self.result = "Computer Wins!"
    else:
        self.result = "Draw"

def _handle_goals(self):
    """
    Detect goals and update the appropriate player's score.

```

A goal is awarded only after the puck has completely passed through the goal opening.

```
"""
```

```
# Left goal: Computer scores.
```

```
if self.puck.x + self.puck.radius < 0:
```

```
    if GOAL_TOP < self.puck.y < GOAL_BOTTOM:
```

```
        self.computer_score += 1
```

```
        self._reset_puck()
```

```
    else:
```

```
        self.puck.x = MARGIN + self.puck.radius
```

```
        self.puck.vx = -self.puck.vx
```

```
# Right goal: Player scores.
```

```
elif self.puck.x - self.puck.radius > WIDTH:
```

```
    if GOAL_TOP < self.puck.y < GOAL_BOTTOM:
```

```
        self.player_score += 1
```

```
        self._reset_puck()
```

```
    else:
```

```
        self.puck.x = WIDTH - MARGIN - self.puck.radius
```

```
        self.puck.vx = -self.puck.vx
```

```
def _reset_puck(self):
```

```
    """
```

```
    Reset the puck to the center.
```

Task 4 will change this so the puck immediately launches again after a goal.

```
"""
```

```
self.puck.x = WIDTH / 2
```

```
self.puck.y = HEIGHT / 2
```

```
self.puck.vx = 0
```

```
self.puck.vy = 0
```

```
def get_winner(self):
```

```
    """
```

```
    Return the current/final match result.
```

```
    Before the match ends, return None.
```

```
    """
```

```
    if not self.match_over:
```

```
        return None
```

```
    return self.result
```

```
def is_match_over(self):
    """Return True once the 30-second match has ended."""
    return self.match_over

def draw(self, surface, font):
    from game import renderer

    renderer.draw_table(surface)

    # Task 2: score display.
    renderer.draw_score(
        surface,
        font,
        self.player_score,
        self.computer_score,
    )

    # Task 3: countdown timer.
    renderer.draw_timer(
        surface,
        font,
        self.match_time_remaining,
    )

    renderer.draw_paddle(
        surface,
        self.player,
        renderer.COLOR_PLAYER,
    )

    renderer.draw_paddle(
        surface,
        self.computer,
        renderer.COLOR_COMPUTER,
    )

    renderer.draw_puck(
        surface,
        self.puck,
    )

    # Task 3: final result.
    if self.match_over:
```

```

        renderer.draw_result(
            surface,
            font,
            self.result,
        )
    )
Complete updated renderer.py
"""

renderer: all pygame drawing lives here, kept separate from game logic.
"""

```

```

import pygame

```

```

WIDTH, HEIGHT = 800, 500
MARGIN = 20

```

```

GOAL_HEIGHT = 150
GOAL_TOP = HEIGHT / 2 - GOAL_HEIGHT / 2
GOAL_BOTTOM = HEIGHT / 2 + GOAL_HEIGHT / 2

```

```

COLOR_BG = (15, 15, 25)
COLOR_TABLE = (20, 60, 90)
COLOR_WALL = (200, 200, 210)
COLOR_CENTER_LINE = (90, 130, 150)
COLOR_PUCK = (240, 240, 240)
COLOR_PLAYER = (60, 140, 240)
COLOR_COMPUTER = (240, 80, 80)
COLOR_TEXT = (255, 255, 255)

```

```

WINDOW_SIZE = (WIDTH, HEIGHT)

```

```

def draw_table(surface):
    surface.fill(COLOR_BG)

    pygame.draw.rect(
        surface,
        COLOR_TABLE,
        (
            MARGIN,
            MARGIN,
            WIDTH - 2 * MARGIN,
            HEIGHT - 2 * MARGIN,
        ),
    )

```

```
pygame.draw.line(
    surface,
    COLOR_CENTER_LINE,
    (WIDTH / 2, MARGIN),
    (WIDTH / 2, HEIGHT - MARGIN),
    2,
)
```

Top and bottom walls.

```
pygame.draw.rect(
    surface,
    COLOR_WALL,
    (0, 0, WIDTH, MARGIN),
)
```

```
pygame.draw.rect(
    surface,
    COLOR_WALL,
    (0, HEIGHT - MARGIN, WIDTH, MARGIN),
)
```

Left wall sections, leaving the goal opening.

```
pygame.draw.rect(
    surface,
    COLOR_WALL,
    (
        0,
        MARGIN,
        MARGIN,
        GOAL_TOP - MARGIN,
    ),
)
```

```
pygame.draw.rect(
    surface,
    COLOR_WALL,
    (
        0,
        GOAL_BOTTOM,
        MARGIN,
        HEIGHT - MARGIN - GOAL_BOTTOM,
    ),
)
```

```

# Right wall sections, leaving the goal opening.
pygame.draw.rect(
    surface,
    COLOR_WALL,
    (
        WIDTH - MARGIN,
        MARGIN,
        MARGIN,
        GOAL_TOP - MARGIN,
    ),
)

pygame.draw.rect(
    surface,
    COLOR_WALL,
    (
        WIDTH - MARGIN,
        GOAL_BOTTOM,
        MARGIN,
        HEIGHT - MARGIN - GOAL_BOTTOM,
    ),
)

def draw_puck(surface, puck):
    pygame.draw.circle(
        surface,
        COLOR_PUCK,
        (int(puck.x), int(puck.y)),
        puck.radius,
    )

def draw_paddle(surface, paddle, color):
    pygame.draw.circle(
        surface,
        color,
        (int(paddle.x), int(paddle.y)),
        int(paddle.radius),
    )

def draw_score(surface, font, player_score, computer_score):

```

```
"""Draw both players' scores at the top of the play area."""
```

```
player_text = font.render(  
    f"Player: {player_score}",  
    True,  
    COLOR_TEXT,  
)
```

```
computer_text = font.render(  
    f"Computer: {computer_score}",  
    True,  
    COLOR_TEXT,  
)
```

```
player_rect = player_text.get_rect(  
    midtop=(WIDTH * 0.25, 2)  
)
```

```
computer_rect = computer_text.get_rect(  
    midtop=(WIDTH * 0.75, 2)  
)
```

```
surface.blit(player_text, player_rect)  
surface.blit(computer_text, computer_rect)
```

```
def draw_timer(surface, font, time_remaining):
```

```
    """Draw the remaining match time."""
```

```
    seconds_remaining = max(  
        0,  
        int(time_remaining + 0.999),  
    )
```

```
    timer_text = font.render(  
        f"Time: {seconds_remaining}",  
        True,  
        COLOR_TEXT,  
    )
```

```
    timer_rect = timer_text.get_rect(  
        midtop=(WIDTH / 2, 2)  
    )
```

```
surface.blit(timer_text, timer_rect)
```

```
def draw_result(surface, font, result):
```

```
    """Display the final match result."""
```

```
    surf = font.render(
        result,
        True,
        (255, 220, 80),
    )
```

```
    rect = surf.get_rect(
        center=(
            surface.get_width() // 2,
            surface.get_height() // 2,
        )
    )
```

```
    surface.blit(surf, rect)
```

```
def draw_text(surface, font, text, pos, color=COLOR_TEXT):
```

```
    surface.blit(
        font.render(text, True, color),
        pos,
    )
```

```
def draw_banner(surface, font, text):
```

```
    surf = font.render(
        text,
        True,
        (255, 220, 80),
    )
```

```
    rect = surf.get_rect(
        center=(
            surface.get_width() // 2,
            surface.get_height() // 2,
        )
    )
```

```
    surface.blit(surf, rect)
```

What should happen now

At match start:

Player: 0 Time: 30 Computer: 0

During play the timer decreases according to actual elapsed time:

Player: 2 Time: 17 Computer: 1

At exactly 30 seconds:

puck stops

AI stops

player movement stops

timer displays 0

scores remain visible

result is displayed

For example:

Player: 2 Time: 0 Computer: 1

Player Wins!

Or, if scores are equal:

Player: 2 Time: 0 Computer: 2

Draw

One important Task 3 detail

I intentionally did not change `_reset_puck()` to launch the puck again. Your current Task 2 behavior resets it to the center and stops it, and changing that is specifically Task 4. This keeps each task isolated and avoids accidentally mixing the next requirement into Task 3.

Your working Task 1 collision implementation should remain untouched. The uploaded original `puck.py`, for example, is the pre-Task-1 version, so don't revert to it.

Sources

task3 worked successfully, provide me with the changes for task4

Task 4 only needs to change `game_engine.py`.

The goal is: after a goal is scored, the puck returns to the center and immediately starts moving again, while keeping all Task 1–3 behavior unchanged.

Exact changes

Keep the existing `_handle_goals()` logic.

Change `_reset_puck()` so it:

moves the puck to the center,

resets its previous position,

immediately calls `_launch_puck()`.

Do not reset the timer or scores.

Do not modify `puck.py`, `collisions.py`, or `renderer.py`.

Complete updated `game_engine.py`

"""

GameEngine: owns the puck, both paddles, and the computer AI, and runs one frame's worth of game logic.

Task 4 adds immediate puck relaunch after every goal.

"""

import random

import pygame

from game.puck import Puck

from game.paddle import Paddle

from game.ai import ComputerAI

from game.collisions import handle_paddle_collision

from game.renderer import WIDTH, HEIGHT, MARGIN, GOAL_TOP, GOAL_BOTTOM

PLAYER_SPEED = 6

PUCK_RADIUS = 12

PADDLE_RADIUS = 28

INITIAL_PUCK_SPEED = 4.5

MATCH_DURATION = 30.0

class GameEngine:

def __init__(self):

self.puck = Puck(WIDTH / 2, HEIGHT / 2, PUCK_RADIUS)

self._launch_puck()

self.player = Paddle(

x=WIDTH * 0.15,

```

        y=HEIGHT / 2,
        radius=PADDLE_RADIUS,
        min_x=MARGIN + PADDLE_RADIUS,
        max_x=WIDTH / 2 - PADDLE_RADIUS,
        min_y=MARGIN + PADDLE_RADIUS,
        max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
    )

    self.computer = Paddle(
        x=WIDTH * 0.85,
        y=HEIGHT / 2,
        radius=PADDLE_RADIUS,
        min_x=WIDTH / 2 + PADDLE_RADIUS,
        max_x=WIDTH - MARGIN - PADDLE_RADIUS,
        min_y=MARGIN + PADDLE_RADIUS,
        max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
    )

    self.ai = ComputerAI()

    # Task 2: scores.
    self.player_score = 0
    self.computer_score = 0

    # Task 3: match timer/state.
    self.match_start_time = pygame.time.get_ticks()
    self.match_time_remaining = MATCH_DURATION
    self.match_over = False
    self.result = None

    def _launch_puck(self):
        angle_choices = [0.3, 0.6, -0.3, -0.6]
        direction = random.choice([-1, 1])
        vy_factor = random.choice(angle_choices)

        self.puck.vx = INITIAL_PUCK_SPEED * direction
        self.puck.vy = INITIAL_PUCK_SPEED * vy_factor

    def handle_input(self, keys_pressed):
        # Do not allow player movement after the match has ended.
        if self.match_over:
            return

        dx = dy = 0

```

```

if keys_pressed[pygame.K_UP]:
    dy -= PLAYER_SPEED

if keys_pressed[pygame.K_DOWN]:
    dy += PLAYER_SPEED

if keys_pressed[pygame.K_LEFT]:
    dx -= PLAYER_SPEED

if keys_pressed[pygame.K_RIGHT]:
    dx += PLAYER_SPEED

self.player.move_by(dx, dy)

def update(self):
    # Once the match has ended, the entire game state is frozen.
    if self.match_over:
        return

    self._update_timer()

    # The timer may have expired during _update_timer().
    if self.match_over:
        return

    self.ai.update(self.computer, self.puck)

    self.puck.move()
    self.puck.bounce_off_walls(HEIGHT, MARGIN)

    # Task 1 collision handling remains unchanged.
    handle_paddle_collision(self.puck, self.player)
    handle_paddle_collision(self.puck, self.computer)

    self._handle_goals()

def _update_timer(self):
    """
    Update the remaining match time using real elapsed time.

    pygame.time.get_ticks() is used instead of frame counting so the
    match lasts 30 real seconds regardless of the frame rate.
    """

```

```

current_time = pygame.time.get_ticks()

elapsed_seconds = (
    current_time - self.match_start_time
) / 1000.0

self.match_time_remaining = max(
    0.0,
    MATCH_DURATION - elapsed_seconds,
)

if self.match_time_remaining <= 0.0:
    self.match_time_remaining = 0.0
    self._end_match()

def _end_match(self):
    """Stop the match and determine its final result."""
    if self.match_over:
        return

    self.match_over = True

    # Stop the puck so nothing continues moving visually after time expires.
    self.puck.vx = 0.0
    self.puck.vy = 0.0

    if self.player_score > self.computer_score:
        self.result = "Player Wins!"
    elif self.computer_score > self.player_score:
        self.result = "Computer Wins!"
    else:
        self.result = "Draw"

def _handle_goals(self):
    """
    Detect goals and update the appropriate player's score.

    A goal is awarded only after the puck has completely passed through
    the goal opening.
    """

    # Left goal: Computer scores.
    if self.puck.x + self.puck.radius < 0:
        if GOAL_TOP < self.puck.y < GOAL_BOTTOM:

```

```

        self.computer_score += 1
        self._reset_puck()
    else:
        self.puck.x = MARGIN + self.puck.radius
        self.puck.vx = -self.puck.vx

# Right goal: Player scores.
elif self.puck.x - self.puck.radius > WIDTH:
    if GOAL_TOP < self.puck.y < GOAL_BOTTOM:
        self.player_score += 1
        self._reset_puck()
    else:
        self.puck.x = WIDTH - MARGIN - self.puck.radius
        self.puck.vx = -self.puck.vx

def _reset_puck(self):
    """
    Reset the puck to the center and immediately launch it again.

    Task 4: the puck no longer remains stationary after a goal.
    """

    # Place the puck exactly at the center.
    self.puck.x = WIDTH / 2
    self.puck.y = HEIGHT / 2

    # Reset the previous position as well so Task 1's swept collision
    # detection starts from the new center position.
    self.puck.previous_x = self.puck.x
    self.puck.previous_y = self.puck.y

    # Immediately give the puck a new clean direction and velocity.
    self._launch_puck()

def get_winner(self):
    """
    Return the current/final match result.

    Before the match ends, return None.
    """
    if not self.match_over:
        return None

    return self.result

```

```

def is_match_over(self):
    """Return True once the 30-second match has ended."""
    return self.match_over

def draw(self, surface, font):
    from game import renderer

    renderer.draw_table(surface)

    # Task 2: score display.
    renderer.draw_score(
        surface,
        font,
        self.player_score,
        self.computer_score,
    )

    # Task 3: countdown timer.
    renderer.draw_timer(
        surface,
        font,
        self.match_time_remaining,
    )

    renderer.draw_paddle(
        surface,
        self.player,
        renderer.COLOR_PLAYER,
    )

    renderer.draw_paddle(
        surface,
        self.computer,
        renderer.COLOR_COMPUTER,
    )

    renderer.draw_puck(
        surface,
        self.puck,
    )

    # Task 3: final result.
    if self.match_over:

```

```
        renderer.draw_result(
            surface,
            font,
            self.result,
        )
Files changed
File      Change
game_engine.py    Updated _reset_puck() to immediately relaunch the puck
puck.py           No change
collisions.py     No change
renderer.py       No change
Task 4 behavior
```

After a goal:

```
Goal detected
↓
Score increases
↓
Puck moves to center
↓
previous_x / previous_y reset to center
↓
New velocity/direction generated
↓
Play immediately continues
```

The 30-second timer keeps running, and the scores are preserved. If the timer reaches zero, the existing Task 3 end-of-match logic still takes over and freezes the game.

the restart R in the game never worked since the start, what should i do to fix that?

The cause is clear from the files available: there is currently no R-key restart handling in the uploaded game code. The GameEngine has no restart method, and none of the files contains a `pygame.K_r` handler.

Your current GameEngine is also where the score and 30-second timer live, so the clean restart behavior should be:

Press R → create a fresh GameEngine → scores reset to 0–0 → timer resets to 30 → puck relaunched → paddles/AI reset.

I don't want to guess at your main game loop, because that is almost certainly where the R key needs to be handled. The files you've provided are game_engine.py, renderer.py, puck.py, paddle.py, ai.py, and collisions.py; there is no main.py/game-loop file among them.
game_engine

What I recommend

Your main loop should contain logic approximately like this:

```
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False

    elif event.type == pygame.KEYDOWN:
        if event.key == pygame.K_r:
            game = GameEngine()
```

The important part is:

```
game = GameEngine()
```

rather than trying to manually reset individual variables. That gives you a genuinely fresh match, including:

- Player score → 0
- Computer score → 0
- Timer → 30 seconds
- match_over → False
- Result → None
- Puck → center with a new launch
- Player paddle → starting position
- Computer paddle → starting position
- AI state → freshly initialized

I need your main game-loop file to give you the exact complete corrected file. Upload the file that contains pygame.event.get(), pygame.display, and the main while running: loop.

Sources